

Taller: Diseño estratégico de casos de prueba

Objetivo

Para cada ejercicio, el equipo debe:

1. Identificar las reglas de negocio.
2. Identificar condiciones, límites y estados.
3. Seleccionar las técnicas de diseño de pruebas apropiadas.
4. Definir el mínimo de casos de prueba necesarios.
5. Justificar por qué utilizaron esas técnicas.

Importante: No se busca crear la mayor cantidad de casos, sino obtener una cobertura adecuada utilizando técnicas de diseño.

Ejercicio 1 – Sistema de pagos

Una aplicación permite realizar pagos con tarjeta.

Reglas de negocio

- El monto máximo permitido es **\$10.000**.
- La tarjeta debe estar **activa**.
- El usuario debe estar **autenticado**.
- Si el monto es **mayor a \$5.000**, se debe solicitar un **OTP**.
- El usuario tiene máximo **3 intentos fallidos de OTP**.
- Después de 3 intentos fallidos, el usuario queda **bloqueado**.

Ejercicio 1 – Sistema de pagos

Reto

Identifiquen :

- ¿Qué **particiones de equivalencia** existen?
- ¿Qué **valores frontera** deberían probar?
- ¿Qué combinaciones requieren una **tabla de decisión**?
- ¿Qué escenarios corresponden a **pruebas negativas**?
- ¿Cuál sería el **mínimo conjunto de casos** que deberían ejecutar?

Ejercicio 2 – Reserva de vuelo y concurrencia

Una aerolínea tiene 1 único asiento disponible.

Dos o más usuarios pueden intentar comprarlo simultáneamente.

Reglas

- El asiento solamente puede ser vendido a un usuario.
- Una vez seleccionado, el usuario tiene 10 minutos para realizar el pago.
- Si el pago no se completa dentro de los 10 minutos, el asiento debe volver a estar disponible.
- Si otro usuario intenta comprar el mismo asiento mientras está reservado, debe recibir un mensaje indicando que no está disponible.

Ejercicio 2 – Reserva de vuelo y concurrencia

Reto

Identifiquen:

- ¿Qué escenarios deben probar?
- ¿Qué sucede si dos usuarios intentan comprar simultáneamente?
- ¿Qué valores frontera utilizarían para los 10 minutos?
- ¿Qué pruebas negativas deberían realizar?
- ¿Cuántos casos consideran necesarios?

Agreguen escenarios como:

- Pago antes de los 10 minutos.
- Pago exactamente en los 10 minutos.
- Pago después de los 10 minutos.
- Usuario cierra el navegador.
- Se pierde la conexión durante el pago.
- Dos usuarios intentan comprar simultáneamente.

Ejercicio 3 – Registro de usuario

Una plataforma permite crear una cuenta.

Reglas

El sistema debe validar:

- El email debe ser único.
- La contraseña debe tener mínimo 8 caracteres.
- La contraseña debe contener al menos un número.
- La contraseña debe contener al menos una letra mayúscula.
- El usuario debe tener mínimo 18 años.
- Los campos obligatorios no pueden quedar vacíos.

Ejercicio 3 – Registro de usuario

Reto

Identifiquen:

- Particiones de equivalencia.
- Valores frontera.
- Casos negativos.
- Combinaciones de reglas que deberían probar.
- Mínimo número de casos necesarios.

Ejercicio 3 – Registro de usuario

Valores que deberían considerar

Edad:

- 17
- 18
- 19

Contraseña:

- 7 caracteres
- 8 caracteres
- 9 caracteres

Además:

- Sin número.
- Sin mayúscula.
- Email ya registrado.
- Campo obligatorio vacío.
- Datos válidos.

Ejercicio 4 – Flujo de estados

Un documento tiene el siguiente flujo:

BORRADOR



EN REVISIÓN



APROBADO



PUBLICADO

Pero existe una regla adicional:

EN REVISIÓN



RECHAZADO

Ejercicio 4 – Flujo de estados

Reglas

- Un documento nuevo comienza en Borrador.
- Desde Borrador puede pasar a En revisión.
- Desde En revisión puede:
 - Aprobarse.
 - Rechazarse.
- Un documento aprobado puede pasar a Publicado.
- No se deben permitir transiciones no definidas.

Ejercicio 4 – Flujo de estados

Reto

Construyan:

1. El diagrama de estados.
2. Las transiciones válidas.
3. Las transiciones inválidas.
4. Los casos de prueba necesarios.

Estado actual

Borrador

En revisión

En revisión

Aprobado

Acción

Enviar a revisión

Aprobar

Rechazar

Publicar

Resultado esperado

En revisión

Aprobado

Rechazado

Publicado

Ejercicio 4 – Flujo de estados

Pregunta

¿Qué casos probarían para asegurarse de que un documento no pueda pasar directamente de Borrador a Publicado?

Ejercicio 5 – API de órdenes

Existe el siguiente EndPoint

POST /api/orders

Para crear una orden:

- Se requiere un token de autenticación.
- El body debe ser un JSON válido.
- El producto debe tener stock disponible.
- Una orden no debe crearse dos veces si el cliente reenvía la misma solicitud.

Ejemplo :

```
{  
  "productId": 123,  
  "quantity": 2  
}
```

Ejercicio 5 – API de órdenes

Reto

Identifiquen pruebas para:

Autenticación

- Token válido.
- Token inválido.
- Token ausente.
- Token expirado.

JSON

- JSON válido.
- JSON mal formado.
- Campos obligatorios ausentes.
- Tipos de datos incorrectos.

Ejercicio 5 – API de órdenes

Reto

Identifiquen pruebas para:

Autenticación

- Token válido.
- Token inválido.
- Token ausente.
- Token expirado.

JSON

- JSON válido.
- JSON mal formado.
- Campos obligatorios ausentes.
- Tipos de datos incorrectos.

Stock

- Stock suficiente.
- Stock exactamente igual a la cantidad solicitada.
- Stock insuficiente.
- Producto sin stock.

Duplicidad

¿Qué debería suceder si el cliente envía exactamente la misma solicitud dos veces?

Ejercicio 5 – API de órdenes

Ejemplo

Request 1 → Crear orden

Request 2 → Misma solicitud

La expectativa debería definirse antes de escribir el caso.

Reto final

¿Cuántos casos de prueba mínimos propondrían y qué técnica utilizarían para cada grupo?

Preguntas

¿Una sola técnica es suficiente para probar un sistema real?

Si todos nuestros casos de prueba pasan, ¿podemos afirmar que el sistema funciona correctamente?

¿Qué otras cosas necesitaríamos validar?

¿El objetivo de QA es encontrar la mayor cantidad de errores posible o reducir el riesgo de que el software falle donde realmente importa?